

VOCALIS: A Lightweight Neural Adapter for Synthetic-to-Natural Speaker Embedding Adaptation

Balaji Ganesh Tammireddy

School of Computer Science and Engineering
Artificial Intelligence and Machine Learning
VIT-AP University
Amaravati, Andhra Pradesh, India
tammireddybalaji@gmail.com

Puruhutika Rao Tata

School of Computer Science and Engineering
Artificial Intelligence and Machine Learning
VIT-AP University
Amaravati, Andhra Pradesh, India
puruhutikarao@gmail.com

Abstract—Zero-shot speech synthesis has reached a point where synthetic speech is now used alongside natural recordings throughout speech processing pipelines. This creates a problem that has received little direct attention: speaker embeddings extracted from synthetic speech do not necessarily occupy the same region of representation space as embeddings extracted from natural recordings, since pretrained speaker encoders are built and tuned on natural speech in the first place. We present VOCALIS, a small residual adapter that learns to map a synthetic speaker embedding toward its natural counterpart, without touching the pretrained encoder or any downstream system. The adapter is trained on paired natural and synthetic speech built from the VCTK corpus, using a speaker-disjoint split of 28 training, 6 validation, and 6 test speakers, with 45 utterances retained per speaker. On the held-out test speakers, cosine similarity between synthetic and natural embeddings rises from 0.0081 with no adaptation to 0.3239 after adaptation. This exceeds both a mean-shift baseline (0.2051) and a Deep CORAL baseline (0.1747) evaluated under the same protocol. The result holds up across five random training seeds (0.3240 ± 0.0015), and the trained adapter — despite never seeing gTTS-generated audio during training — still improves cosine similarity from 0.0507 to 0.3165 when tested on gTTS output. The adapter itself is small: about 462,000 parameters, trained typically in 2–3 minutes, and never more than about five, on ordinary CPU hardware. We state these limitations plainly: the absolute similarity achieved is still well below 1.0, the test set is only 6 speakers, and no downstream task (voice conversion, verification) has been evaluated directly. Within that scope, the results indicate that a lightweight, purely embedding-space adapter is a workable way to narrow the synthetic-to-natural gap.

I. INTRODUCTION

Speaker embeddings are one of the quiet workhorses of modern speech technology. A text-to-speech system that needs to sound like a particular person, a voice conversion pipeline, a diarization system trying to tell two speakers apart in a meeting recording — all of them lean on a pretrained encoder that turns a chunk of audio into a fixed-length vector carrying speaker identity and, ideally, little else. These encoders are trained almost exclusively on natural, human-recorded speech, because that is what was available when GE2E [1], x-vectors [2], and later ECAPA-TDNN [3] were developed.

At the same time, synthetic speech has gotten good enough, and cheap enough, that it now shows up on both sides of the same pipelines these encoders were built for — as training

data, as evaluation data, sometimes as both. That creates a question worth asking directly: when you run a piece of TTS-generated audio through an encoder that has only ever seen real voices, does the resulting embedding actually land in the same part of the vector space as the equivalent natural recording? Or does it end up somewhere systematically different, shifted by whatever the synthesis process adds — prosodic flatness, spectral artifacts, the particular acoustic fingerprint of a given TTS engine?

Most of the recent progress in this general area has gone toward making the synthetic speech itself better, or making the embeddings themselves more discriminative, rather than asking whether the two are actually compatible with each other. Domain adaptation as a field has plenty to say about aligning mismatched distributions in general [4], [5], but very little of that work has been pointed specifically at the synthetic-versus-natural speaker embedding gap.

This paper is about that gap. We built VOCALIS, a small residual network that sits entirely inside embedding space: it takes a synthetic speaker embedding as input and learns to output something closer to what a natural recording of the same speaker and words would have produced. It does not touch the speaker encoder, and it does not generate or modify any audio. The only thing being trained is the adapter itself, using paired natural and synthetic embeddings as supervision — same speaker, same transcript, different acoustic source.

We wanted the evaluation to hold up under scrutiny, not just show a before/after number. So alongside the main result, we built in a speaker-disjoint train/validation/test split, compared against two classical linear baselines (a simple mean-shift and Deep CORAL), checked whether the trained adapter still works on a TTS system it never saw during training, retrained the whole thing across five random seeds to see if the result was stable, and looked at how performance changes as the training set shrinks. All of that is reported here, including the parts that came out less clean than we expected — the mean-shift baseline, for instance, actually beats full CORAL in our setting, and we say so rather than hiding it.

The contributions of this paper are:

- We frame the synthetic-to-natural speaker embedding gap as a concrete, testable domain adaptation problem, rather

than a side effect to be assumed away.

- We introduce VOCALIS, a lightweight residual adapter (about 462,000 parameters) that learns this mapping directly from paired embeddings, without retraining the speaker encoder or any downstream model.
- We evaluate it under a speaker-disjoint protocol against linear baselines, across multiple random seeds, on a synthesis system unseen during training, and across varying amounts of training data — reporting results honestly, including where they were closer than expected.
- We show that the adapter improves cosine similarity from 0.0081 to 0.3239 on held-out test speakers, a result that holds within 0.5% variability across five seeds and generalizes to gTTS-synthesized audio the model never trained on.

II. RELATED WORK

A. Speaker Representation Learning

The move toward compact, fixed-length speaker embeddings has a fairly clear lineage. GE2E [1] and x-vectors [2] established the idea that a neural network trained on a speaker classification or verification objective could be repurposed to produce a general-purpose embedding, usable for tasks the network was never explicitly trained on. ECAPA-TDNN [3] pushed this further with channel attention and multi-scale feature aggregation, and remains one of the more widely adopted architectures for this purpose. A recent comparative study looks specifically at how ECAPA-TDNN and x-vector embeddings behave in zero-shot multi-speaker TTS settings [6], and self-supervised models like WavLM [7] have shown that a single pretrained representation can transfer across a surprising range of downstream speech tasks.

What this body of work has in common is that it is almost entirely concerned with natural speech, both for training the encoders and for evaluating them. Whether the same encoders behave consistently when the input is synthetic rather than recorded is a question that sits just outside the scope of most of this research, which is exactly the gap we are trying to address.

B. Zero-Shot Speech Synthesis

Zero-shot TTS has moved fast. YourTTS [8] showed that a single model could handle multilingual synthesis and voice conversion for speakers it had never seen, conditioned only on a short reference clip. Since then, NaturalSpeech 3 [9], VALL-E 2 [10], StyleTTS-ZS [11], and DS-TTS [12] have each pushed naturalness and speaker similarity further, using neural codec language modeling, diffusion, and various forms of style conditioning.

A separate thread of this literature looks at lightweight adapters specifically for improving synthesis quality — parameter-efficient modules bolted onto a larger TTS backbone rather than full retraining [13]–[15]. This is close in spirit to what we are doing architecturally (a small trainable module layered on top of something larger and frozen), but the target is different: these systems adapt the synthesis model to improve

the audio it produces, whereas we are adapting the embedding extracted *from* already-synthesized audio, after the fact, without touching generation at all.

Across this entire body of work, speaker embeddings are treated as an input signal to condition the synthesis process, not as something that itself needs correction once synthetic audio exists.

C. Domain Adaptation for Speech Representations

Domain adaptation as a general problem — reducing the mismatch between a source and target distribution while keeping the information that actually matters for the task — has a long history outside speech. Deep CORAL [4] does this by matching second-order covariance statistics between domains; DANN [5] instead trains a representation to fool a domain classifier, pushing source and target features together adversarially. Both have been reused extensively in vision and language tasks.

Within speech specifically, related ideas show up in cross-lingual voice transfer and in conditioning schemes meant to be robust across different synthesis settings [16]. These are domain adaptation problems in a loose sense, but the domains being aligned are languages or recording conditions, not synthetic-versus-natural speech in the way we are framing it here.

We use CORAL directly as a baseline in this paper rather than only citing it, since it is the most directly comparable classical method to what VOCALIS is trying to do: align a source distribution (synthetic embeddings) to a target distribution (natural embeddings) using paired data.

D. Where This Leaves Us

Put the three threads together and there is a real gap in the middle: speaker representation learning keeps improving the encoders, zero-shot synthesis keeps improving the audio, and domain adaptation has the tools to align mismatched distributions — but almost nothing sits at the intersection of all three, asking whether synthetic-speech embeddings and natural-speech embeddings, extracted by the same encoder, actually agree with each other. VOCALIS is our attempt to occupy that specific space: not a new encoder, not a new TTS system, just a small learned correction that operates after both of those pipelines have already done their job.

III. METHODOLOGY

A. Problem Definition

Take a synthetic speech utterance and its embedding $\mathbf{e}_s \in \mathbb{R}^d$, produced by some pretrained speaker encoder. Take the same speaker reading the same transcript, but recorded naturally, and call its embedding $\mathbf{e}_n \in \mathbb{R}^d$. If the encoder treated synthetic and natural speech identically, $\mathbf{e}_s$ and $\mathbf{e}_n$ would land close together. In practice they do not (we show exactly how far apart in Section V). What we want is a function

$$f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^d \quad (1)$$

such that

$$f_{\theta}(\mathbf{e}_s) \approx \mathbf{e}_n. \quad (2)$$

We keep the encoder itself completely fixed. θ is the only thing being learned, and it lives entirely downstream of the encoder, in embedding space. This is a deliberate constraint, not an oversight: it means VOCALIS can be dropped into any pipeline that already has a working speaker encoder, without retraining that encoder or touching whatever system consumes its output.

B. Framework Overview

Figure 1 lays out the full pipeline. Natural recordings come from VCTK; a matching synthetic recording is generated offline for each one, using the same transcript. Both go through the same speaker encoder, producing a pair of embeddings that differ only because one came from a human and the other from a TTS system.

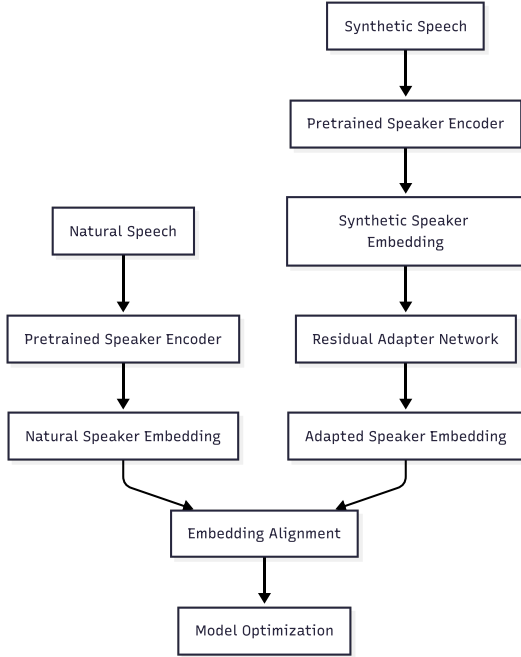


Fig. 1. Overview of the VOCALIS framework. Natural and synthetic speech are passed through the same frozen speaker encoder. The synthetic embedding is refined using the proposed residual adapter network before alignment with the corresponding natural speaker embedding during training.

During training, $\mathbf{e}_s$ is the input and $\mathbf{e}_n$ is the target. At inference time, there is no natural recording available at all — that is the whole point. You have a synthetic embedding, you pass it through the trained adapter, and what comes out is meant to sit closer to where the natural version would have been.

C. Dataset Construction

Natural speech comes from VCTK [17]. Speakers are split into training, validation, and test sets with no overlap between them — a speaker who shows up in training never shows up in validation or test. This is enforced before any embeddings

are even extracted, at the point where speaker IDs are assigned to a split.

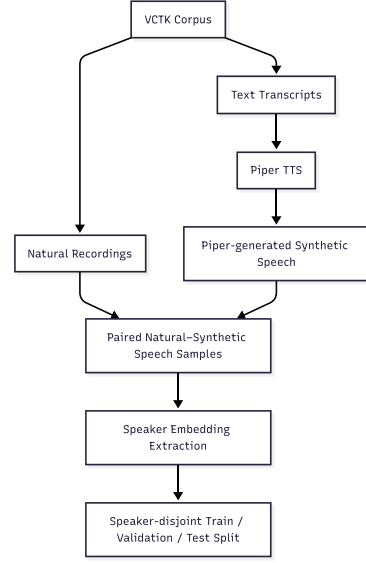


Fig. 2. Construction of the paired natural–synthetic speech dataset.

For every kept utterance, a synthetic version is generated offline with Piper [18], using the exact transcript from the natural recording. The two recordings say the same thing; they just sound different. Everything — speaker ID, transcript, which split a speaker belongs to, file paths for both the natural and synthetic audio — is tracked in a metadata table, so that any later stage of the pipeline can be traced back to exactly which recording it came from.

Before any training happens, we run a check over the whole dataset: no speaker appears in two splits, every utterance that should have both a natural and synthetic recording actually has both, and embedding files exist where the metadata says they should. This catches problems before they turn into a wasted training run.

The overall dataset construction procedure is illustrated in Fig. 2.

D. Speaker Embedding Extraction

Embeddings are extracted from both the natural and synthetic recordings using a pretrained ECAPA-TDNN encoder [3]. The implementation also supports Resemblyzer as an alternative encoder, but every result reported in this paper uses ECAPA-TDNN; we did not run a full comparison against Resemblyzer, and we say so rather than implying otherwise.

The embedding dimensionality is read directly from the loaded encoder at run time rather than hardcoded anywhere, so the same code works whichever encoder is active. Every embedding is computed once, during preprocessing, and saved to disk — training never touches raw audio again after this point, which is both faster and removes any inconsistency that could come from re-extracting embeddings on the fly.

E. Residual Adapter Network

The adapter itself is small on purpose. It takes a d -dimensional embedding and passes it through two hidden layers (each Linear $\rightarrow$ LayerNorm $\rightarrow$ ReLU $\rightarrow$ Dropout), then a final linear layer back down to d dimensions. That output is treated as a learned correction, not the final answer directly — it gets blended with the original input through a single learnable scalar α :

$$\hat{\mathbf{e}}_s = \text{L2Norm}(\mathbf{e}_s + \alpha \cdot g_\theta(\mathbf{e}_s)) \quad (3)$$

where g_θ is the two-layer MLP described above. At $\alpha = 0$ this is exactly the identity mapping (with L2 normalization); at $\alpha = 1$ the residual pass-through and the learned correction are weighted equally before normalization. α is not fixed — it is learned along with everything else, and Section V reports where it ends up.

Only the adapter’s parameters are updated during training. The encoder stays frozen throughout. In total the adapter has about 462,000 trainable parameters and trains in typically 2–3 minutes, and never more than about five, on ordinary CPU hardware, with no GPU involved anywhere in the process. The final output is L2-normalized before being returned, which matters because cosine similarity — our main evaluation metric — is exactly the right comparison for unit-length vectors.

F. Training Objective

Three terms make up the loss:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{cos}} + \lambda_2 \mathcal{L}_{\text{mse}} + \lambda_3 \mathcal{L}_{\text{reg}} \quad (4)$$

The cosine term pushes the adapted embedding toward the same direction as the natural target:

$$\mathcal{L}_{\text{cos}} = 1 - \frac{\hat{\mathbf{e}}_s^T \mathbf{e}_n}{\|\hat{\mathbf{e}}_s\| \|\mathbf{e}_n\|} \quad (5)$$

The MSE term additionally pushes it toward the same position, not just the same direction:

$$\mathcal{L}_{\text{mse}} = \|\hat{\mathbf{e}}_s - \bar{\mathbf{e}}_n\|_2^2, \quad \bar{\mathbf{e}}_n = \frac{\mathbf{e}_n}{\|\mathbf{e}_n\|_2} \quad (6)$$

We normalize $\mathbf{e}_n$ here specifically because $\hat{\mathbf{e}}_s$ is always unit-length by construction — comparing a unit vector against a raw, unnormalized embedding would make this loss term dominated by scale rather than actual similarity, which is not what we want it measuring. Both sides need to live in the same normalized space for the comparison to mean anything.

The third term, $\mathcal{L}_{\text{reg}}$, penalizes the adapted embedding for drifting too far from the original synthetic input. Its job is to stop the network from finding a shortcut where every synthetic embedding just gets mapped toward some average natural-looking point, regardless of which speaker it actually came from — a failure mode that is easy to fall into if nothing keeps the transformation anchored to its input.

G. Training Procedure

Gradient updates only ever come from the training split. Model selection — which checkpoint we actually keep — is based on validation loss, not training loss, and the test split is never looked at until evaluation. We use Adam, mini-batch updates, and check validation loss after every epoch; whichever epoch has the lowest validation loss is the one that gets saved and used for every downstream experiment in this paper (held-out testing, cross-synthesis, seed sweeps, the residual weight ablation, and the training-size sweep all use this same checkpoint, or a freshly retrained equivalent where the experiment specifically calls for retraining).

H. Inference Pipeline

At inference time, none of the training machinery is needed. Take a synthetic recording, run it through the same encoder used during training, pass the resulting embedding through the trained adapter, and what comes out is the adapted embedding. That is the entire output of VOCALIS — a vector, not audio. Whatever system consumes that vector downstream (a voice conversion model, a verification system, anything else that expects a speaker embedding) receives it exactly as it would receive any other embedding, with no changes required on its end.

IV. EXPERIMENTAL SETUP

A. Dataset

All experiments use the VCTK speech corpus [17], which contains multi-speaker English recordings across a range of regional accents. Speakers are split into training, validation, and test sets of 28, 6, and 6 speakers respectively, with no speaker appearing in more than one split. We keep 45 utterances per speaker across all three sets, so the number of paired samples per speaker stays constant regardless of which split they fall into. Every natural recording has a synthetic counterpart built from the same transcript, so the only thing that differs between a pair is the acoustic source. Table I summarizes this configuration alongside the rest of the experimental setup.

TABLE I
EXPERIMENTAL CONFIGURATION USED THROUGHOUT THE EVALUATION.

Parameter	Value
Dataset	VCTK Corpus
Training / Validation / Test Speakers	28 / 6 / 6
Utterances per Speaker	45
Synthetic Speech Engine	Piper
Cross-Synthesis Evaluation	gTTS
Speaker Encoder	ECAPA-TDNN
Adapter Parameters	~462,000
Optimizer	Adam
Model Selection	Best Validation Checkpoint
Primary Metric	Cosine Similarity

B. Synthetic Speech Generation

Synthetic audio is generated offline with Piper [18], one utterance at a time, using the transcript already attached to the corresponding VCTK recording. Piper was chosen mainly for practical reasons: it runs entirely offline on CPU, produces the same output every time given the same input, and does not depend on an external API that could rate-limit or change behavior between runs — all of which matter for a dataset this pipeline needs to regenerate reliably.

To check whether the adapter generalizes past the one synthesis system it was trained on, we separately generate gTTS audio for the test speakers’ transcripts and run the trained adapter on the resulting embeddings, with no retraining or fine-tuning involved. The adapter never sees a single gTTS-derived embedding until this evaluation step.

C. Speaker Embedding Extraction

Embeddings for both natural and synthetic audio are produced by a pretrained ECAPA-TDNN encoder. The pipeline is also written to support Resemblyzer as a drop-in alternative encoder, and embedding dimensionality is inferred automatically from whichever encoder is active rather than fixed in advance — but the results in this paper all come from ECAPA-TDNN specifically; we have not run the Resemblyzer path end-to-end for comparison, and leave that for later work.

Every embedding, natural and synthetic, is extracted exactly once and cached to disk before any training begins. The adapter never sees raw audio — only these cached vectors.

D. Baseline Methods

To check whether a nonlinear adapter is actually earning its keep, we compare VOCALIS against the raw, unadapted embedding as a reference floor, plus two classical linear alternatives fit on the training partition and evaluated on the same held-out test speakers as VOCALIS:

- **Identity mapping (no adaptation)** — the raw synthetic embedding, used as-is, with nothing fit at all. This is the floor everything else is measured against.
- **Mean-shift** — translate the synthetic embedding distribution so its mean matches the natural embedding distribution’s mean. Nothing more sophisticated than that.
- **Deep CORAL** [4] — align second-order covariance structure between the two distributions, not just the mean.

All three, plus VOCALIS, are scored under the same L2-normalized cosine similarity protocol so the comparison is apples-to-apples.

E. Evaluation Metrics

Cosine similarity between the adapted (or baseline) embedding and the true natural embedding is the main number we report throughout. It is the natural choice here since it is scale-invariant and directly answers the question we care about: does the adapted embedding point in essentially the same direction as the real natural embedding. We report both an aggregate number across all test utterances and a per-speaker breakdown, since an aggregate alone can hide a result that only works for some speakers.

F. Experimental Protocol

Beyond the core held-out evaluation, we ran four additional checks meant to stress-test the result rather than just report it once: comparison against the linear baselines above, evaluation on gTTS audio the model never trained on, retraining across five random seeds to check stability, and retraining on progressively smaller subsets of the training speakers to see whether performance actually depends on having more data. Each of these is reported in full in Section V, including the parts of the results that turned out less clean than we expected going in.

V. RESULTS

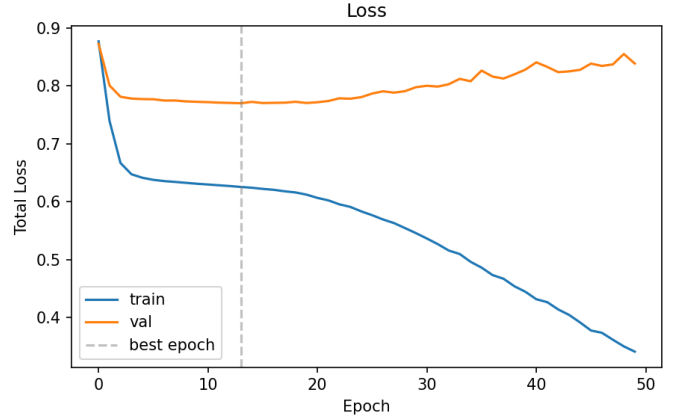


Fig. 3. Training and validation loss during optimization. The dashed vertical line indicates the checkpoint selected using the minimum validation loss.

A. Domain Gap Reduction on Held-Out Speakers

Table II shows what happens on the 6 held-out test speakers (270 utterances total) with and without VOCALIS. Without any adaptation, synthetic and natural embeddings barely correlate at all — mean cosine similarity is 0.0081, close enough to zero that the encoder might as well be looking at unrelated audio. After adaptation, that number rises to 0.3239. Euclidean distance and MSE move the same direction.

TABLE II
AGGREGATE TEST-SET PERFORMANCE: NO ADAPTATION VS. VOCALIS.

Metric	No Adaptation	VOCALIS	Δ
Cosine Similarity $\uparrow$	0.0081	0.3239	+0.3158
Euclidean Distance $\downarrow$	1.4077	1.1601	−0.2476
MSE $\downarrow$	0.0103	0.0070	−0.0033

Table III breaks this down speaker by speaker. All six test speakers improve; none get worse. The size of the improvement varies fairly widely though, from 0.2328 up to 0.4305, which we take to mean that some speakers’ synthetic and natural recordings simply differ more than others under this particular encoder, not that the adapter is working inconsistently.

TABLE III
PER-SPEAKER COSINE SIMILARITY ON HELD-OUT TEST SPEAKERS.

Speaker	Baseline	VOCALIS	Δ	Std
p253	-0.0247	0.4058	+0.4305	0.0532
p257	0.0245	0.4276	+0.4032	0.0795
p281	-0.0260	0.2178	+0.2438	0.0491
p317	0.0120	0.2449	+0.2328	0.0579
p340	0.0122	0.3504	+0.3382	0.0726
p351	0.0504	0.2970	+0.2465	0.0601

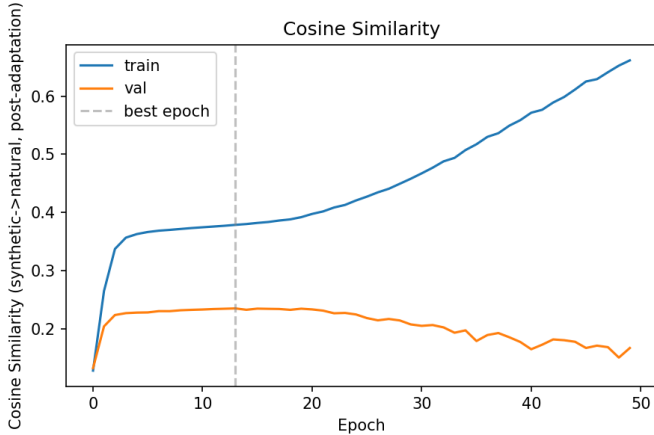


Fig. 4. Evolution of cosine similarity during training. The dashed line marks the checkpoint selected for evaluation.

Six speakers is not a lot to draw statistical conclusions from, so we were careful about how we tested this. Utterances from the same speaker are not independent of each other, so the right unit to test at is the speaker, not the utterance. Running a one-sided sign test at the speaker level, with all 6 of 6 speakers improving, gives $p = (1/2)^6 = 0.0156$, under the usual 0.05 cutoff. It is a small sample and we say so plainly, but the test itself is done correctly.

B. Comparison with Baseline Adaptation Methods

Table IV lines VOCALIS up against the two linear baselines. Both mean-shift (0.2051) and CORAL (0.1747) do meaningfully better than doing nothing, which tells us a good chunk of the gap is just a simple distribution shift that linear methods can partly correct. VOCALIS still comes out ahead of both by a clear margin, 0.1188 over mean-shift and 0.1492 over CORAL, so whatever the adapter is learning goes beyond what a closed-form linear transform can capture with this amount of data.

TABLE IV
COMPARISON OF ADAPTATION METHODS ON HELD-OUT TEST SPEAKERS.

Method	Mean Cosine Similarity
No Adaptation	0.0081
Mean-Shift	0.2051
Deep CORAL [4]	0.1747
VOCALIS (proposed)	0.3239

One result here is worth flagging rather than glossing over: the simpler mean-shift baseline beats full CORAL, even though CORAL does strictly more (covariance alignment on top of mean alignment). We think this is most likely a data-size issue — estimating a full covariance matrix reliably needs more paired examples than estimating a mean does, and at 1260 training pairs the covariance estimate may just be noisier than it would be with more data. We return to this in Section VI.

C. Reproducibility Across Random Seeds

One good result from one lucky initialization is not worth much, so we retrained the whole adapter five times from scratch, changing only the random seed, and evaluated each one on the same held-out test speakers.

TABLE V
TEST COSINE SIMILARITY ACROSS 5 RANDOM SEEDS.

Seed	Test Cosine Similarity	Improvement
42	0.3223	+0.3143
123	0.3261	+0.3180
7	0.3234	+0.3153
2024	0.3249	+0.3169
99	0.3235	+0.3154
Mean $\pm$ Std	0.3240 $\pm$ 0.0015	+0.3160 $\pm$ 0.0015

The five runs land between 0.3223 and 0.3261, with a standard deviation of only 0.0015 — about 0.5% of the mean. That is small enough that we are comfortable calling 0.324 a stable number for this dataset, not something that happened to come out well on one particular run.

D. Cross-Synthesis Generalization

The adapter only ever saw Piper audio during training. Table VI shows what happens when we hand it gTTS audio instead, for the same test speakers, with no retraining at all.

TABLE VI
CROSS-SYNTHESIS GENERALIZATION: PIPER (TRAINED ON) VS. GTTS (UNSEEN).

Synthesis System	Baseline Cosine	Adapted Cosine
Piper (in-distribution)	0.0081	0.3239
gTTS (unseen)	0.0507	0.3165

Cosine similarity on gTTS audio goes from 0.0507 to 0.3165, close to the Piper number, even though the model never trained on a single gTTS embedding. All 6 test speakers improve here too, by between 0.1595 and 0.3644. We also notice that the unadapted baseline is near-zero for *both* synthesis systems (0.0081 for Piper, 0.0507 for gTTS), which points toward the domain gap being a general property of synthetic speech under this encoder, rather than something specific to Piper’s particular acoustic signature.

E. Residual Weight Analysis

The adapter’s blending scalar α (Section III) converged to 0.9696 during training. Table VII shows what happens to test cosine similarity if we manually override α after the fact, using the same trained network.

TABLE VII
TEST COSINE SIMILARITY AS A FUNCTION OF RESIDUAL WEIGHT α .

Residual Weight (α)	Test Cosine Similarity
0.00 (identity mapping)	0.0081
0.25	0.1982
0.50	0.2853
0.75	0.3143
0.97 (as trained)	0.3239
1.00	0.3247

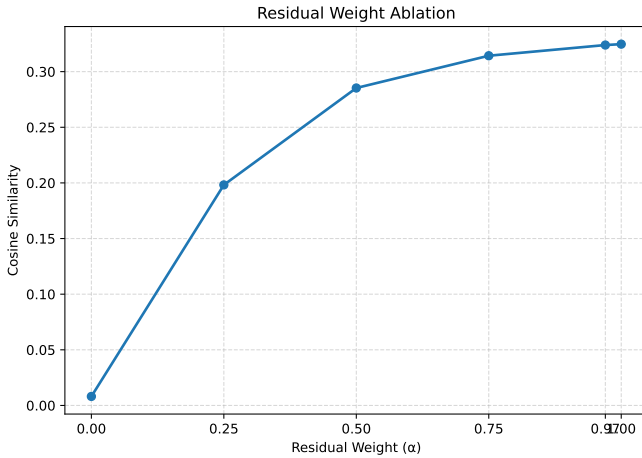


Fig. 5. Effect of the residual blending weight α on cosine similarity. Performance improves steadily as greater emphasis is placed on the learned nonlinear correction.

At $\alpha = 0$ the number matches the no-adaptation baseline in Table II exactly, a useful sanity check that the residual connection behaves the way it is supposed to. What is more interesting is the other end: fixing $\alpha = 1.0$ (no residual pass-through of the original input at all) actually gives a slightly *higher* score, 0.3247, than the value the network converged to on its own, 0.9696. In plain terms, the residual pass-through is not doing much useful work here; almost all of the improvement comes from the learned nonlinear correction itself.

F. Effect of Training Data Scale

We retrained the adapter on shrinking subsets of the 28 training speakers (28, 14, 7, and 3), keeping the same fixed 6-speaker test set throughout, to see whether the result depends on having a reasonably large training set or holds up even with very little data.

Performance climbs steadily as more speakers are added, with the biggest jump between 3 and 7 and progressively smaller gains after that. This looks like a genuine learning curve rather than a result that only holds at one particular

TABLE VIII
TEST COSINE SIMILARITY AS A FUNCTION OF TRAINING SPEAKER COUNT.

Train Speakers	Train Pairs	Test Cosine Similarity
3	135	0.2664
7	315	0.3041
14	630	0.3183
28	1260	0.3223

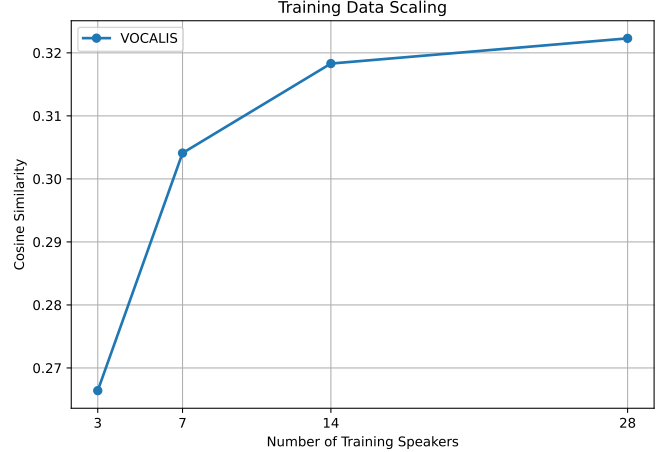


Fig. 6. Effect of training set size on adaptation performance. Test cosine similarity consistently improves as the number of training speakers increases, indicating that the proposed adapter benefits from additional paired training data and has not yet reached performance saturation.

dataset size, and it suggests there is still room to improve simply by training on more speakers than the 28 used here.

VI. DISCUSSION

The main thing these results tell us is that the synthetic-to-natural embedding gap is not a small effect or an artifact of one particular encoder choice — a cosine similarity of 0.0081 with no adaptation means the encoder is essentially treating synthetic and natural recordings of the same speaker as unrelated. Every evaluation we ran, across different test speakers, different random seeds, and a completely different TTS system, points the same direction: VOCALIS closes a real chunk of that gap, and does so consistently rather than on a lucky run.

The comparison against linear baselines is worth sitting with for a moment. Both mean-shift and CORAL improve on doing nothing at all, which means a meaningful part of this gap really is just a shift in where the two distributions sit — nothing exotic. But VOCALIS still beats both by a wide margin, which tells us the remaining structure is not something a closed-form linear transform can pick up, at least not with the amount of paired data available here. The fact that mean-shift actually outperforms the more sophisticated CORAL is the kind of result we would have been tempted to leave out, but it is real and worth explaining rather than hiding: CORAL needs to estimate a full covariance matrix, and with only 1260 training pairs feeding a 192-dimensional embedding space, that

estimate is likely noisier than the much simpler mean estimate mean-shift relies on. This is not a knock against CORAL as a method, just a reminder that its data requirements are steeper.

The gTTS result matters more than it might look at first glance. VOCALIS was never shown a single gTTS-derived embedding during training, and it still recovers most of its performance on gTTS audio. If the adapter had simply memorized something specific to how Piper’s voice model sounds, we would expect this number to collapse. It does not. That suggests the model picked up on something closer to a general property of synthetic speech under this encoder, rather than a quirk of one TTS system.

The residual weight result is the one place where our design assumption did not quite hold up the way we expected. We built in a learnable blend between the original synthetic embedding and the network’s correction specifically so the model could fall back toward the input if the correction was not helping. In practice, the model learned to almost fully ignore that fallback ($\alpha = 0.9696$), and forcing α to exactly 1.0 does marginally better still. At this data scale, the residual connection is not pulling its weight. We are not removing it from the architecture on the strength of one dataset size, since it is possible this changes with more training data or a different loss weighting, but we would rather report this plainly than describe the residual connection as more important to the result than the numbers actually show.

The training-size sweep is the most encouraging result for anyone thinking about scaling this up: performance keeps climbing as more speakers are added, and the curve has not flattened out by 28 speakers. That is a fairly direct argument that this is a real learned pattern rather than something that happens to work at exactly this dataset size, and it means the natural next step is simply more data, not a different architecture.

We want to be equally direct about what this study does not show. Every number reported here comes from English speech, from one corpus, using one primary encoder. We have not tested whether an improved embedding actually translates into a better outcome on a downstream task — a voice conversion system, a speaker verification pipeline — that actually consumes these embeddings; that is a real gap, and cosine similarity in isolation is a proxy, not a guarantee. The test set is 6 speakers out of the 110 available in VCTK, and while the sign test and the scaling experiment both support treating this as a genuine effect rather than noise, a larger and more varied speaker pool would make that case more convincingly. And we checked cross-synthesis generalization on exactly one unseen TTS system (gTTS); how this holds up against, say, a neural codec-based system like the ones cited in Section II is an open question.

None of that changes what we think the core finding is: a small, fixed-encoder adapter, trained on paired embeddings alone, can recover a substantial and reproducible fraction of the synthetic-to-natural gap, and do so cheaply enough to run on a laptop CPU in a few minutes. Whether that translates into a meaningfully better voice conversion or TTS pipeline in practice is the next thing to check, not something we are claiming here.

VII. CONCLUSION

We set out to answer a fairly specific question: do speaker embeddings extracted from synthetic speech actually agree with embeddings extracted from natural speech, when both go through the same pretrained encoder. The answer, at least under ECAPA-TDNN and on the VCTK-derived data used here, is no — not without help. Cosine similarity between the two sits at 0.0081 by default, which is about as disconnected as two representations of the same speaker’s voice can be.

VOCALIS is our attempt at closing that gap without touching anything upstream or downstream of the embedding itself. It is a small residual network, about 462,000 parameters, trained only on paired synthetic and natural embeddings, and it recovers cosine similarity up to 0.3239 on speakers it never saw during training. That result held up across five random seeds, beat both linear baselines we tested it against, and carried over reasonably well to a TTS system (gTTS) it was never trained on. A residual-weight ablation and a training-size sweep both supported treating this as a real, generalizable effect rather than a coincidence of one particular run.

We have tried to be equally clear about where this stops. The absolute similarity achieved is still well short of 1.0 — this narrows the gap, it does not eliminate it. The evaluation stays entirely inside the embedding space; we have not shown that a better embedding actually improves a downstream voice conversion or verification system, only that it should be more compatible with one. The test set is small, and everything here is English speech from a single corpus.

Those limitations point fairly directly at what comes next: running the adapted embeddings through an actual downstream system to see if the embedding-space improvement holds up where it would actually matter, testing on a larger and more diverse speaker pool, and checking whether the same approach transfers to other languages and to newer synthesis systems beyond Piper and gTTS. None of that requires rethinking the approach — the architecture is small enough and the training cheap enough that scaling any of these dimensions up is mostly a matter of more data and more compute time, not a new method.

If there is a broader point here, it is that the gap between synthetic and natural speaker representations is not something that has to be accepted as a fixed cost of using synthetic speech in a pipeline. A small, cheaply trained module sitting entirely in embedding space recovered a substantial fraction of that gap in our experiments. Whether that generalizes further is an open question, but it seems like a question worth continuing to ask.

REFERENCES

- [1] L. Wan, Q. Wang, A. Papir, and I. L. Moreno, “Generalized end-to-end loss for speaker verification,” *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 4879–4883, 2018.
- [2] D. Snyder, D. Garcia-Romero, G. Sell, D. Povey, and S. Khudanpur, “X-vectors: Robust dnn embeddings for speaker recognition,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2018, pp. 5329–5333.

- [3] B. Desplanques, J. Thienpondt, and K. Demuynck, "Ecapa-tdnn: Emphasized channel attention, propagation and aggregation in tdnns based speaker verification," in *Proceedings of Interspeech 2020*, 2020, pp. 3830–3834.
- [4] B. Sun and K. Saenko, "Deep coral: Correlation alignment for deep domain adaptation," in *Proceedings of the European Conference on Computer Vision (ECCV) Workshops*, 2016, pp. 443–450.
- [5] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, and V. Lempitsky, "Domain-adversarial training of neural networks," vol. 17, no. 59, 2016, pp. 1–35. [Online]. Available: <https://jmlr.org/papers/v17/15-239.html>
- [6] M. Kunešová, Z. Hanzlíček, and J. Matoušek, "An exploration of ecapa-tdnn and x-vector speaker representations in zero-shot multi-speaker tts," *arXiv preprint arXiv:2506.20190*, 2025. [Online]. Available: <https://arxiv.org/abs/2506.20190>
- [7] S. Chen, C. Wang, Z. Chen, Y. Wu, S. Liu, Z. Chen, J. Li, N. Kanda, T. Yoshioka, X. Xiao, and L. Zhou, "Wavlm: Large-scale self-supervised pre-training for full stack speech processing," *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 6, pp. 1505–1518, 2022.
- [8] E. Casanova, J. Weber, C. Shulby, A. Candido Junior, E. Gölge, and M. A. Ponti, "Yourtts: Towards zero-shot multi-speaker tts and zero-shot voice conversion for everyone," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 2709–2720. [Online]. Available: <https://proceedings.mlr.press/v162/casanova22a.html>
- [9] Z. Ju, Y. Wang, K. Shen, X. Tan, D. Xin, D. Yang, Y. Liu, Y. Leng, K. Song, S. Tang, Z. Wu, T. Qin, X.-Y. Li, W. Ye, S. Zhang, J. Bian, L. He, J. Li, and S. Zhao, "Naturalspeech 3: Zero-shot speech synthesis with factorized codec and diffusion models," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 22 605–22 623. [Online]. Available: <https://proceedings.mlr.press/v235/ju24b.html>
- [10] S. Chen, S. Liu, L. Zhou, Y. Liu, X. Tan, J. Li, S. Zhao, Y. Qian, and F. Wei, "Vall-e 2: Neural codec language models are human parity zero-shot text to speech synthesizers," *arXiv preprint arXiv:2406.05370*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.05370>
- [11] Y. A. Li, X. Jiang, C. Han, and N. Mesgarani, "Styletts-zs: Efficient high-quality zero-shot text-to-speech synthesis with distilled time-varying style diffusion," *arXiv preprint arXiv:2409.10058*, 2024. [Online]. Available: <https://arxiv.org/abs/2409.10058>
- [12] M. Meng, Z. Yang, J. Yang, Z. Su, Y. Zhu, and Z. Fan, "Ds-tts: Zero-shot speaker style adaptation from voice clips via dynamic dual-style feature modulation," *arXiv preprint arXiv:2506.01020*, 2025. [Online]. Available: <https://arxiv.org/abs/2506.01020>
- [13] K. Fujita, H. Sato, T. Ashihara, H. Kanagawa, M. Delcroix, T. Moriya, and Y. Ijima, "Noise-robust zero-shot text-to-speech synthesis conditioned on self-supervised speech-representation model with adapters," *arXiv preprint arXiv:2401.05111*, 2024. [Online]. Available: <https://arxiv.org/abs/2401.05111>
- [14] K. Fujita, T. Ashihara, H. Kanagawa, T. Moriya, and Y. Ijima, "Zero-shot text-to-speech synthesis conditioned using self-supervised speech representation model," *arXiv preprint arXiv:2304.11976*, 2023. [Online]. Available: <https://arxiv.org/abs/2304.11976>
- [15] K. Fujita, T. Ashihara, M. Delcroix, and Y. Ijima, "Lightweight zero-shot text-to-speech with mixture of adapters," in *Proceedings of Interspeech 2024*, 2024. [Online]. Available: https://www.isca-archive.org/interspeech_2024/fujita24b_interspeech.html
- [16] F. Biadys, Y. Chen, I. Elias, K. Kastner, G. Wang, A. Rosenberg, and B. Ramabhadran, "Zero-shot cross-lingual voice transfer for tts," *arXiv preprint arXiv:2409.13910*, 2024. [Online]. Available: <https://arxiv.org/abs/2409.13910>
- [17] J. Yamagishi, C. Veaux, and K. MacDonald, "Cstr vctk corpus: English multi-speaker corpus for cstr voice cloning toolkit (version 0.92)," 2019.
- [18] Rhasspy Project, "Piper: A fast, local neural text-to-speech system," GitHub repository, 2023, accessed: 2026-07-06. [Online]. Available: <https://github.com/rhasspy/piper>